```
!pip install -q transformers torch accelerate pandas pyarrow
```

```
import torch
import re
import pandas as pd
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
dataset = load_dataset("gsm8k", "main")
data = dataset["test"].to_pandas()

print("Total samples:", len(data))
print(data.head())
```

```
Total samples: 1319
                                            question  \
0  Janet’s ducks lay 16 eggs per day. She eats th...
1  A robe takes 2 bolts of blue fiber and half th...
2  Josh decides to try flipping a house.  He buys...
3  James decides to run 3 sprints 3 times a week....
4  Every day, Wendi feeds each of her chickens th...

                                              answer
0  Janet sells 16 - 3 - 4 = <<16-3-4=9>>9 duck eg...
1  It takes 2/2=<<2/2=1>>1 bolt of white fiber\nS...
2  The cost of the house and repairs came out to ...
3  He sprints 3*3=<<3*3=9>>9 times\nSo he runs 9*...
4  If each chicken eats 3 cups of feed per day, t...
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cpu
```

```
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32
).to(device)

model.eval()
```

```
config.json: 100%  608/608 [00:00<00:00, 18.8kB/s]
tokenizer_config.json: 1.29k/? [00:00<00:00, 22.5kB/s]
tokenizer.json: 1.84M/? [00:00<00:00, 24.4MB/s]
tokenizer.model: 100%  500k/500k [00:00<00:00, 1.79MB/s]
special_tokens_map.json: 100%  551/551 [00:00<00:00, 15.9kB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%  2.20G/2.20G [00:30<00:00, 213MB/s]
Loading weights: 100%  201/201 [00:09<00:00, 30.10it/s, Materializing param=model.norm.weight]
generation_config.json: 100%  124/124 [00:00<00:00, 4.50kB/s]
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 2048)
    (layers): ModuleList(
      (0-21): 22 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)
```

```
def build_prompt(question, mode="zero_shot", exemplars=None):

    if mode == "zero_shot":
        return (
            "Solve the following math problem.\n"
            "Give only the final numeric answer.\n\n"
            f"Question: {question}\n"
            "Answer:"
        )

    elif mode == "few_shot":
        prompt = "Here are some solved examples:\n\n"
        for q, a in exemplars:
            prompt += f"Q: {q}\nA: {a}\n\n"

        prompt += (
            "Now solve the following problem.\n\n"
            f"Q: {question}\n"
```

```
            "A:"
        )
        return prompt

    elif mode == "cot":
        return (
            "Solve the following math problem step by step.\n"
            "After reasoning, give the final answer in the format:\n"
            "Answer: <number>\n\n"
            f"Question: {question}\n"
            "Let's think step by step.\n"
        )

    else:
        raise ValueError("Invalid mode")
```

```
def generate_answer(prompt, max_new_tokens=150):

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None
```

```
def get_few_shot_examples(k=3):
    examples = []
    for i in range(k):
        q = data.iloc[i]["question"]
        a = data.iloc[i]["answer"].split("####")[-1].strip()
        examples.append((q, a))
    return examples

few_shot_examples = get_few_shot_examples(3)
```

```
def evaluate_mode(mode, n_samples=20):

    correct = 0
    hallucinations = 0

    for i in range(n_samples):

        question = data.iloc[i]["question"]
        gt_answer = data.iloc[i]["answer"].split("####")[-1].strip()

        if mode == "few_shot":
            prompt = build_prompt(question, mode, few_shot_examples)
```

```
        else:
            prompt = build_prompt(question, mode)

        output = generate_answer(prompt)

        pred = extract_last_number(output)
        gt = extract_last_number(gt_answer)

        if pred is None:
            hallucinations += 1
        elif pred == gt:
            correct += 1

        print(f"[{mode}] Sample {i+1} | Pred: {pred} | GT: {gt}")

    accuracy = correct / n_samples
    hallucination_rate = hallucinations / n_samples

    print("\n----------------------")
    print(f"Mode: {mode}")
    print(f"Accuracy: {accuracy:.3f}")
    print(f"Hallucination Rate: {hallucination_rate:.3f}")
    print("----------------------\n")

    return accuracy, hallucination_rate
```

```
acc_zero, hall_zero = evaluate_mode("zero_shot")
acc_few, hall_few = evaluate_mode("few_shot")
acc_cot, hall_cot = evaluate_mode("cot")
```

```
[zero_shot] Sample 1 | Pred: 120 | GT: 18
[zero_shot] Sample 2 | Pred: 2 | GT: 3
[zero_shot] Sample 3 | Pred: 000 | GT: 70000
[zero_shot] Sample 4 | Pred: 1800 | GT: 540
[zero_shot] Sample 5 | Pred: 40 | GT: 20
[zero_shot] Sample 6 | Pred: 16 | GT: 64
[zero_shot] Sample 7 | Pred: 80 | GT: 260
[zero_shot] Sample 8 | Pred: 1.5 | GT: 160
[zero_shot] Sample 9 | Pred: 12 | GT: 45
[zero_shot] Sample 10 | Pred: 11.20 | GT: 460
[zero_shot] Sample 11 | Pred: 240. | GT: 366
[zero_shot] Sample 12 | Pred: 280. | GT: 694
[zero_shot] Sample 13 | Pred: 7 | GT: 13
[zero_shot] Sample 14 | Pred: 10 | GT: 18
[zero_shot] Sample 15 | Pred: 15 | GT: 60
[zero_shot] Sample 16 | Pred: 500. | GT: 125
[zero_shot] Sample 17 | Pred: 200 | GT: 230
[zero_shot] Sample 18 | Pred: 000. | GT: 57500
[zero_shot] Sample 19 | Pred: 120 | GT: 7
[zero_shot] Sample 20 | Pred: 4 | GT: 6

----------------------
Mode: zero_shot
Accuracy: 0.000
Hallucination Rate: 0.000
----------------------

[few_shot] Sample 1 | Pred: 70000 | GT: 18
[few_shot] Sample 2 | Pred: 2 | GT: 3
[few_shot] Sample 3 | Pred: 000 | GT: 70000
```

```
[few_shot] Sample 4 | Pred: 000 | GT: 540
[few_shot] Sample 5 | Pred: 000 | GT: 20
[few_shot] Sample 6 | Pred: 000 | GT: 64
[few_shot] Sample 7 | Pred: 000 | GT: 260
[few_shot] Sample 8 | Pred: 000 | GT: 160
[few_shot] Sample 9 | Pred: 200 | GT: 45
[few_shot] Sample 10 | Pred: 000 | GT: 460
[few_shot] Sample 11 | Pred: 1000 | GT: 366
[few_shot] Sample 12 | Pred: 000 | GT: 694
[few_shot] Sample 13 | Pred: 000 | GT: 13
[few_shot] Sample 14 | Pred: 1 | GT: 18
[few_shot] Sample 15 | Pred: 100 | GT: 60
[few_shot] Sample 16 | Pred: 100 | GT: 125
[few_shot] Sample 17 | Pred: 10 | GT: 230
[few_shot] Sample 18 | Pred: 000 | GT: 57500
[few_shot] Sample 19 | Pred: 4 | GT: 7
[few_shot] Sample 20 | Pred: 000 | GT: 6

----------------------
Mode: few_shot
Accuracy: 0.000
Hallucination Rate: 0.000
----------------------

[cot] Sample 1 | Pred: 16 | GT: 18
[cot] Sample 2 | Pred: 3. | GT: 3
[cot] Sample 3 | Pred: 000 | GT: 70000
[cot] Sample 4 | Pred: 180 | GT: 540
```